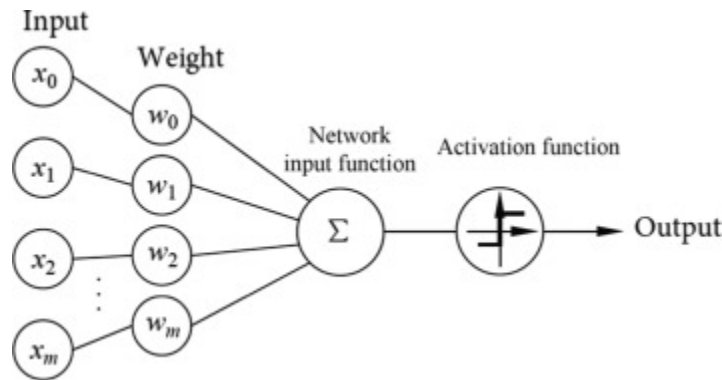


Trabajo Práctico 3: Redes Neuronales



Introducción

En este trabajo vamos a usar la implementación de los MLP estándar que viene en la librería Sklearn. Tanto la versión para problemas de clasificación (MLPClassifier) como para problemas de regresión (MLPRegressor). En el notebook complementario se incluye un ejemplo de uso.

Para los ejercicios del práctico hay que desarrollar una función que entrene la red un número de veces y que mida y devuelva los errores de la misma, para poder hacer curvas de entrenamiento. La función tiene que ser así:

```
#función que entrena una red ya definida previamente "evaluaciones" veces,
cada vez entrenando un número de épocas elegido al crear la red y midiendo
el error en train, validación y test al terminar ese paso de entrenamiento.

#Guarda y devuelve la red en el paso de evaluación que da el mínimo error de
validación

#entradas: la red, las veces que evalúa, los datos de entrenamiento y sus
respuestas, de validacion y sus respuestas, de test y sus respuestas

#salidas: la red entrenada en el mínimo de validación, los errores de train,
validación y test medidos en cada evaluación

def entrenar_red(red, evaluaciones, X_train, y_train, X_val, y_val, X_test,
y_test):

    #mi código
```

Trabajo Práctico 3: Redes Neuronales

```
#red.fit(X_train, y_train)
```

```
#más código
```

```
return best_red, error_train, error_val, error_test
```

Pueden hacer una función para problemas de regresión y una para clasificación, o todo en una. Pueden agregar lo que quieran, ese diseño es para darles una idea de lo que hay que hacer.

Importante: Para hacer una copia de una red ya creada, para mantener la mejor que encontraron hasta el momento, se debe hacer con la función `deepcopy()` de la librería `copy`. Esto asegura que se guardan copias de todos los elementos anidados de la estructura de datos.

La medida de error que vamos a usar en regresión es error cuadrático medio (`sk.metrics.mean_squared_error()`) y para clasificación es el error de clasificación (`sk.metrics.zero_one_loss()`)

Advertencia: El entrenamiento de las redes es computacionalmente pesado. Algunos de los entrenamientos que se plantean en el trabajo llevan varios minutos, dependiendo de la velocidad del procesador. El tiempo máximo en Google Colab fue de alrededor de 10 minutos. Tengan eso en cuenta para hacer el trabajo.

Ejercicios

Entregar un notebook con el código (que incluya la generación de las gráficas) y texto con las discusiones pedidas.

Ejercicio 1. Capacidad de Modelado.

Entrene redes neuronales para resolver el problema de clasificación de las espirales anidadas que creamos en el TP 0. Use un número creciente de neuronas en la capa intermedia: 2, 10, 20, 40. Valores posibles para los demás parámetros de entrenamiento: learning rate 0.1, momentum 0.9, 600 datos para ajustar los modelos (20% de ese conjunto separarlo al azar para conjunto de validación), 2000 para testear, 1000 evaluaciones del entrenamiento, cada una de 20 épocas. Para cada uno de los cuatro modelos obtenidos, graficar en el plano xy las clasificaciones sobre el conjunto de test. Comentar.

Trabajo Práctico 3: Redes Neuronales

Ejercicio 2. Mínimos Locales.

Baje el dataset *dos-elipses* de la descargas. Realice varios entrenamientos con los siguientes parámetros: 6 neuronas en la capa intermedia, 500 patrones en el training set, de los cuales 400 se usan para entrenar y 100 para validar el modelo (sacados del .data), 2000 patrones en el test set (del .test), 300 evaluaciones del entrenamiento, cada una de 50 épocas.

Pruebe distintos valores de momentum y learning-rate (valores usuales son 0, 0.5, 0.9 para el momentum y 0.1, 0.01, 0.001 para el learning-rate, pero no hay por qué limitarse a esos valores), para tratar de encontrar el mejor mínimo posible de la función error. El valor que vamos a usar es el promedio de 10 entrenamientos iguales, dado que los entrenamientos incorporan el azar. Como guía, con los parámetros dados, hay soluciones entre 5% y 6% de error en test, y tal vez mejores. Confeccione una tabla con los valores usados para los parámetros y el resultado en test obtenido (la media de las 10 ejecuciones). Haga una gráfica de error de train, validación y test en función del número de épocas para los valores seleccionados (los mejores valores de eta y alfa).

Ejercicio 3. Regularización

Baje el dataset Ikeda (en todos los problemas de regresión la última columna del dataset es la variable a precedir). Realice varios entrenamientos usando el 95% del archivo .data para entrenar, y el resto para validar. Realice otros entrenamientos cambiando la relación a 75%-25%, y a 50%-50%. En cada caso seleccione un resultado que considere adecuado, y genere gráficas del mse en train, validación y test. Comente sobre los resultados.

Los otros parámetros para el entrenamiento son: learning rate 0.01, momentum 0.9, 2000 datos para testear, 400 evaluaciones del entrenamiento, cada una de 50 épocas, 30 neuronas en la capa oculta.

Ejercicio 4. Regularización (2).

Vamos a usar regularización por penalización, el weight-decay. Hay que tener cuidado con los nombres de los parámetros en este caso. El parámetro que nosotros llamamos gamma en la teoría corresponde en MLP de sklearn al parámetro alpha, mientras que nosotros usamos alfa para el momentum en general. Para activarlo tenemos que usar:

Trabajo Práctico 3: Redes Neuronales

Ejercicio 5. Dimensionalidad.

Repita el punto 4 del Práctico 1, usando ahora redes con 6 unidades en la capa intermedia. Los otros parámetros hay que setearlos adecuadamente, usando como guía los casos anteriores. Genere una gráfica que incluya los resultados de redes y árboles.

(OPCIONAL) Ejercicio 6. Multiclase.

Busque en los datasets de sklearn los archivos del problema *iris* y entrene una red sobre ellos. Tome un tercio de los datos como test, y los dos tercios restantes para entrenar y validar. Ajuste los parámetros de la red de manera conveniente. Realice curvas de entrenamiento, validación y test.

Baje el dataset *Faces*. Entrene una red neuronal para resolver dicho problema, eligiendo adecuadamente todos los parámetros (hay que re-escalar los datos al rango $[0,1]$ antes de usarlos). Muestre curvas de entrenamiento para asegurar la convergencia. Compare los resultados con los citados en Mitchell (pag. 112).

(OPCIONAL) Ejercicio 7. Minibatch.

La implementación de sklearn permite usar minibatches cambiando el parámetro `batch_size`. Realice una comparación de las curvas de aprendizaje para el problema de Sunspots, utilizando batches de longitud 1 y otros dos valores (3 curvas). Comente los resultados.

Parámetros del entrenamiento: 20% de validación, learning rate 0.05, momentum 0.3, 2000 evaluaciones del entrenamiento, cada una de 200 épocas, 6 neuronas en la capa intermedia.

Los valores del minibatch se deben elegir para que en cada etapa de entrenamiento haya varias actualizaciones del gradiente, o sea que no debería ser mayor a 40.